Cloud and Fog Computing for IoT (CS4001)

CIA-1

Question 2: Smart Healthcare Remote Patient Monitoring Platform

Docker image: **https://hub.docker.com/r/shakthe/vitalwatch** (v1, latest)

GitHub repo link: **Shakthe/cfccia**

Submitted by **Shakthevell M [REG NO] IoT-B**

1. System Architecture Diagram

VitalWatch is designed as a five-layer, cloud-native pipeline on **Amazon Web Services (AWS)** that takes raw vital-sign telemetry from a patient's wearable device all the way through to an alert delivered back to that same wearable, so the patient is warned the moment any vital sign becomes dangerous.

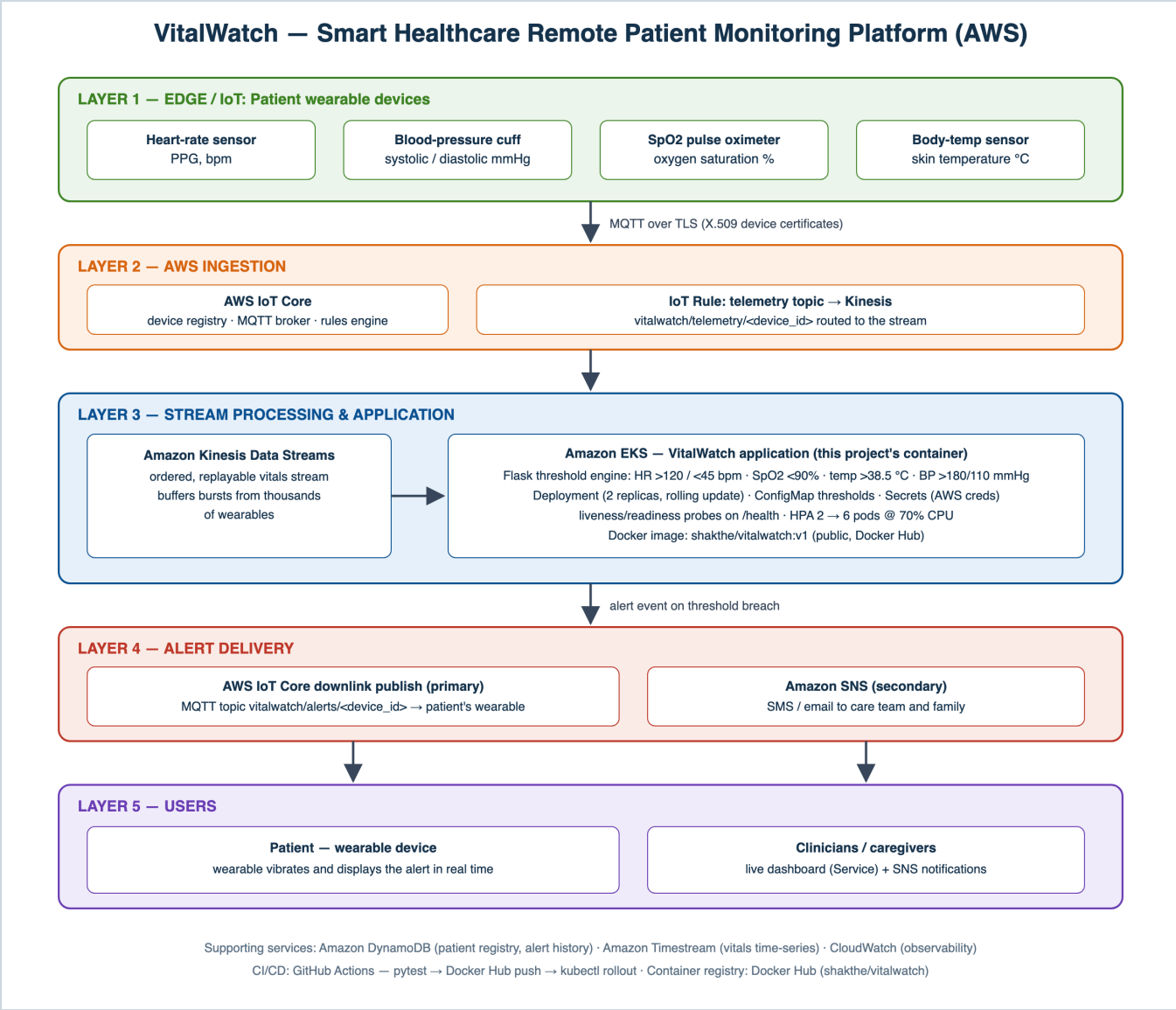


Figure 1.1 — VitalWatch five-layer cloud architecture on AWS

Layer-by-layer flow

Layer	Component	Role
1 — Edge / IoT	Wearable sensors: heart rate (PPG), blood-pressure cuff, SpO2 pulse oximeter, body-temperature sensor	Generates continuous vital-sign telemetry for each registered patient

2 — AWS Ingestion	AWS IoT Core	Device registry, X.509 certificate authentication, and MQTT telemetry ingestion at scale; an IoT Rule routes `vitalwatch/telemetry/<device_id>` into the stream
3 — Stream Processing & App	Amazon Kinesis Data Streams + Amazon EKS (Flask app, ConfigMap/Secrets, HPA)	Evaluates every reading against clinical thresholds (HR >120 or <45 bpm, SpO2 <90%, temperature >38.5 °C, BP >180/110 mmHg) and runs the containerized application
4 — Alert Delivery	AWS IoT Core downlink publish (primary) + Amazon SNS (secondary)	Pushes the alert to the patient's wearable over MQTT topic `vitalwatch/alerts/<device_id>`; SNS notifies the care team by SMS/email
5 — Users	Patient wearable + clinicians/caregivers	The wearable vibrates and displays the alert to the patient; clinicians follow the live dashboard

Supporting services: Amazon DynamoDB (patient registry and alert history), Amazon Timestream (vitals time-series for trend analysis) and CloudWatch (observability).

2. Creating the Docker Image

The application is a Python Flask service (`app/app.py`) that simulates the four wearable vital-sign streams (heart rate, blood pressure, SpO2, body temperature) for every registered patient, evaluates each reading against clinical thresholds (`app/alerts.py`) and pushes alerts to the patient's wearable device over MQTT (simulated when no AWS IoT endpoint is configured, real IoT Core publish when one is).

Step 1 — Write the Dockerfile

The `Dockerfile` defines how the application is packaged:

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /srv/vitalwatch

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

RUN useradd --create-home vitalwatch
USER vitalwatch

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request,sys; \
      sys.exit(0 if urllib.request.urlopen('http://localhost:5000/health').status==200 else
1)"

CMD ["gunicorn", "--workers", "2", "--bind", "0.0.0.0:5000", "app.app:app"]
```

- **Base image:** official lightweight Python 3.12 runtime. The `slim` variant keeps the final image small (213 MB vs ~1 GB for the full image), which speeds up pulls in Kubernetes and reduces attack surface.
- **Environment settings:** disables `.pyc` files and forces real-time log output so `docker logs` / `kubectl logs` show wearable alert pushes immediately.
- **Security best practice:** the container runs as a non-root user (`vitalwatch`), limiting damage in case of compromise.
- **Layer caching optimization:** dependencies are copied and installed before the source code, so editing code never re-downloads packages — rebuilds take seconds.

- **Port + self-healing:** documents port 5000 and lets Docker/Kubernetes detect an unhealthy container automatically via the `/health` endpoint.
- **Production server:** gunicorn (WSGI) instead of Flask's dev server — 2 worker processes handle concurrent requests.

Step 2 — Add a .dockerignore

Excludes tests, git history, Kubernetes manifests, the report and caches from the build context → smaller, cleaner images with no unnecessary files.

Step 3 — Build the image

```
$ docker build -t shakthe/vitalwatch:v1 -t shakthe/vitalwatch:latest .
Step 1/11 : FROM python:3.12-slim
Step 2/11 : ENV PYTHONDONTWRITEBYTECODE=1     PYTHONUNBUFFERED=1
Step 3/11 : WORKDIR /srv/vitalwatch
Step 4/11 : COPY requirements.txt .
Step 5/11 : RUN pip install --no-cache-dir -r requirements.txt
Step 6/11 : COPY app ./app
Step 7/11 : RUN useradd --create-home vitalwatch
Step 8/11 : USER vitalwatch
Step 9/11 : EXPOSE 5000
Step 10/11 : HEALTHCHECK --interval=30s --timeout=5s --retries=3   CMD python -c "import
urllib.request,sys; sys.exit(0 if
urllib.request.urlopen('http://localhost:5000/health').status==200 else 1)"
Step 11/11 : CMD ["gunicorn", "--workers", "2", "--bind", "0.0.0.0:5000", "app.app:app"]
Successfully built 17f8a49fc9c5
Successfully tagged shakthe/vitalwatch:v1
Successfully tagged shakthe/vitalwatch:latest
```

Figure 2.1 — docker build: all 11 steps complete; image tagged v1 and latest

- `-t name:tag` — tags the image with the Docker Hub repository name. Two tags are applied: versioned (`v1`) and rolling (`latest`).
- `.` — build context is the current directory (filtered by `.dockerignore`).

Step 4 — Verify locally

```
$ docker run -d --name vitalwatch -p 5001:5000 shakthe/vitalwatch:v1
793690c48f22caa4147e424d48c67470114f4630c50f41b6268056a929d21533

$ curl -s http://localhost:5001/health
{"monitor_interval_s":15,"patients":3,"status":"healthy"}


$ docker ps --format "table {{.Names}}  {{.Image}}      {{.Status}}     {{.Ports}}"
NAMES                       IMAGE                     STATUS                    PORTS
vitalwatch-control-plane    kindest/node:v1.36.1      Up 2 minutes              127.0.0.1:53055-
>6443/tcp
vitalwatch-local            shakthe/vitalwatch:v1     Up 4 minutes (healthy)    0.0.0.0:5001-
>5000/tcp
```

Figure 2.2 — container running locally with a passing Docker healthcheck ("healthy"); /health returns status healthy

- `-d` detached mode, `-p 5001:5000` maps host port 5001 → container port 5000.
- Verified `/health` returned `{"status": "healthy"}`, patient registration worked, and vital-sign alerts were triggered and dispatched to the wearable topic from inside the container.

Step 5 — Push to Docker Hub

```
docker login -u shakthe
docker push shakthe/vitalwatch:v1
docker push shakthe/vitalwatch:latest
```

The image is now publicly accessible at `docker.io/shakthe/vitalwatch:v1` and can be pulled on any machine or cluster with `docker pull shakthe/vitalwatch:v1`.

[SCREENSHOT TO ADD AFTER docker push — Docker Hub repository page]

Figure 2.3 — public Docker Hub repository shakthe/vitalwatch with the v1 and latest tags

Step 6 — Deploy anywhere without modification

The exact same public image runs unchanged under Kubernetes: `k8s/deployment.yaml` references `image: shakthe/vitalwatch:v1`, and the CI/CD pipeline deploys it into a cluster on every push (Section 4).

3. Kubernetes Deployment Architecture

The Kubernetes manifests (in the `k8s/` directory) deploy the exact same public Docker Hub image without any modification:

Resource	File	Role
Namespace	namespace.yaml	Isolates all VitalWatch resources inside the `vitalwatch` namespace
ConfigMap	configmap.yaml	Clinical thresholds and monitor interval — tunable per environment without rebuilding the image
Secret	secret.yaml	AWS IoT Core endpoint and credentials injected as environment variables (empty in the demo → simulated MQTT publish)
Deployment	deployment.yaml	2 replicas, rolling updates (maxUnavailable=0), liveness/readiness probes on `/health`, resource requests/limits
Service	service.yaml	NodePort 30090 locally — a one-line change to LoadBalancer for EKS
HPA	service.yaml	Autoscales 2 → 6 pods at 70% CPU utilisation

Deployment commands used:

```
kind create cluster --name vitalwatch
kind load docker-image shakthe/vitalwatch:v1 --name vitalwatch
kubectl apply -f k8s/
kubectl rollout status deployment/vitalwatch -n vitalwatch
```

The cluster was verified end-to-end: both replicas are healthy, patient data and vital-sign readings are served through the Service, and the horizontal pod autoscaler is active and watching CPU load.

```
$ kubectl get all -n vitalwatch
NAME                              READY   STATUS    RESTARTS   AGE
pod/vitalwatch-86c4f469dd-4bpvq   1/1     Running   0          30s
pod/vitalwatch-86c4f469dd-rzzr7   1/1     Running   0          30s

NAME                 TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
service/vitalwatch   NodePort   10.96.21.170   <none>        80:30090/TCP   38s

NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/vitalwatch   2/2     2            2           30s

NAME                                    DESIRED   CURRENT   READY   AGE
replicaset.apps/vitalwatch-86c4f469dd   2         2         2       30s

NAME                                                REFERENCE               TARGETS
MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/vitalwatch   Deployment/vitalwatch   cpu: <unknown>/70%
2         6         2          38s
```

Figure 3.1 — kubectl get all -n vitalwatch: 2/2 pods Running, NodePort service, Deployment, ReplicaSet and HPA (2→6 replicas) all healthy

Functional verification through the deployed service

With the Service port-forwarded to `localhost:8080`, the API and dashboard were exercised directly against the running cluster:

```
$ curl -s http://localhost:8080/patients | python3 -m json.tool
[
    {
        "age": 67,
        "condition": "post-cardiac surgery",
        "device_id": "wearable-ak-01",
        "name": "Arun Kumar",
        "patient_id": "P-1001"
    },
    {
        "age": 54,
        "condition": "COPD",
        "device_id": "wearable-mn-02",
        "name": "Meera Nair",
        "patient_id": "P-1002"
    }
]
```

Figure 3.2 — GET /patients returning the registered patients through the cluster's Service

```
$ curl -s -X POST http://localhost:8080/vitals/refresh | python3 -m json.tool
{
    "alerts": [
        {
            "category": "LOW SPO2 ALERT",
            "message": "Oxygen saturation 85% below 90%",
            "patient_id": "P-1001",
            "reading": {
                "diastolic": 76,
                "heart_rate": 73,
                "spo2": 85,
                "systolic": 109,
                "temperature": 36.6
            },
            "timestamp": "2026-08-22T10:12:55Z",
            "topic": "vitalwatch/alerts/wearable-ak-01"
        }
    ],
    "readings": [
        {
            "diastolic": 76,
            "heart_rate": 73,
            "patient_id": "P-1001",
            "spo2": 85,
            "systolic": 109,
            "temperature": 36.6,
            "timestamp": "2026-08-22T10:12:55Z"
        },
        {
            "diastolic": 83,
            "heart_rate": 92,
            "patient_id": "P-1002",
            "spo2": 96,
            "systolic": 127,
            "temperature": 36.9,
            "timestamp": "2026-08-22T10:12:55Z"
        }
    ]
}
```

Figure 3.3 — POST /vitals/refresh forcing a monitor cycle; a LOW SPO2 ALERT is triggered when oxygen saturation drops below the 90% threshold and is pushed to the patient's wearable topic

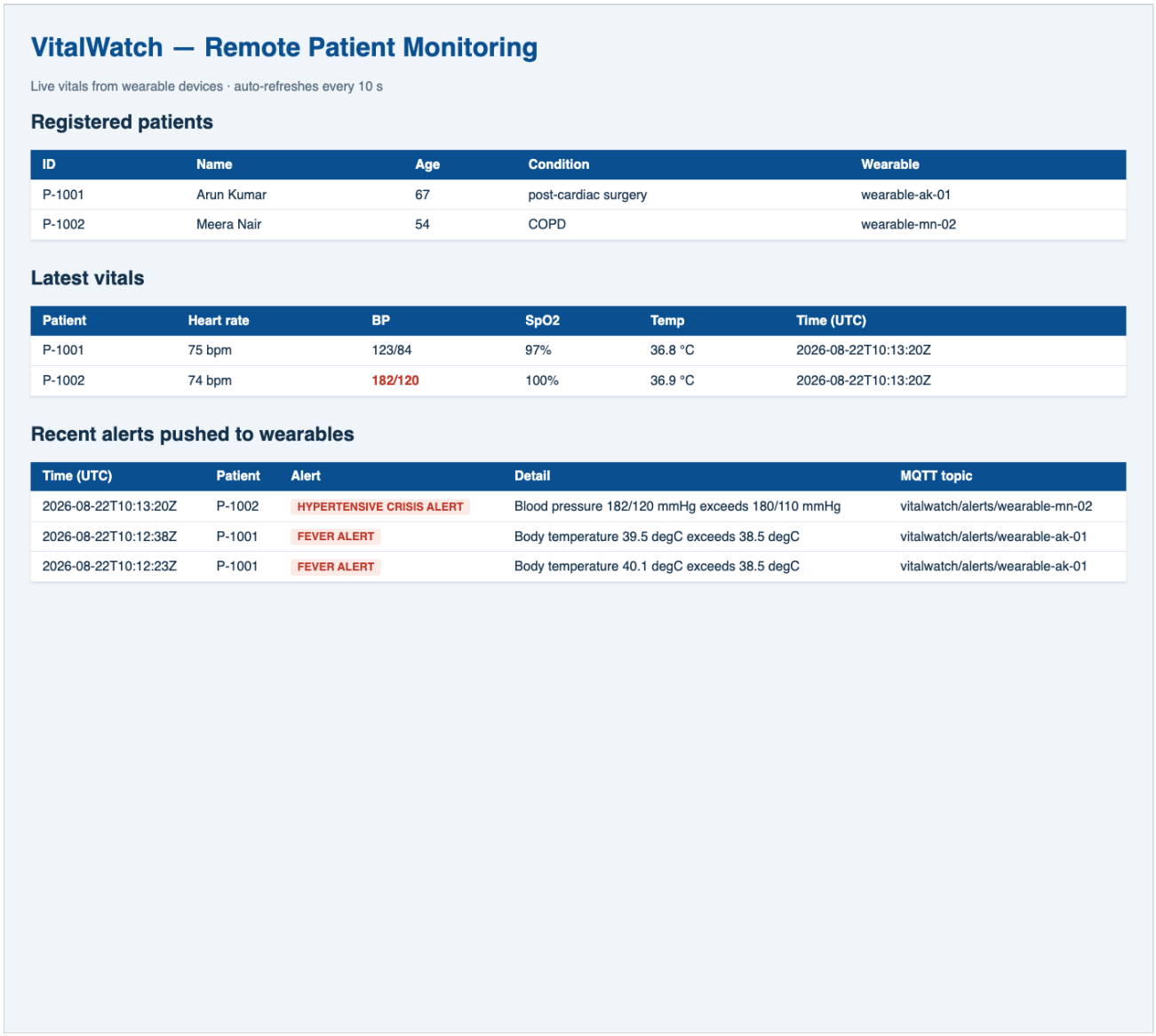


Figure 3.4 — live VitalWatch dashboard served from the cluster showing registered patients, current vitals (breaches highlighted in red) and recent alerts pushed to wearables

The pod logs confirm alerts firing correctly and being pushed to the correct patient's wearable MQTT topic the moment a threshold is breached (the publish is simulated in this environment, and becomes a live AWS IoT Core delivery the moment a real endpoint is set in the Secret):

```
$ kubectl logs vitalwatch-86c4f469dd-4bpvq -n vitalwatch
2026-08-22 10:12:23,057 WARNING vitalwatch.alerts WEARABLE PUSH (simulated MQTT) ->
vitalwatch/alerts/wearable-ak-01 [Arun Kumar] FEVER ALERT: Body temperature 40.1 degC exceeds
38.5 degC
2026-08-22 10:12:38,060 WARNING vitalwatch.alerts WEARABLE PUSH (simulated MQTT) ->
vitalwatch/alerts/wearable-ak-01 [Arun Kumar] FEVER ALERT: Body temperature 39.5 degC exceeds
38.5 degC
2026-08-22 10:12:53,146 WARNING vitalwatch.alerts WEARABLE PUSH (simulated MQTT) ->
vitalwatch/alerts/wearable-mn-02 [Meera Nair] TACHYCARDIA ALERT: Heart rate 151 bpm exceeds
120 bpm
2026-08-22 10:12:55,337 WARNING vitalwatch.alerts WEARABLE PUSH (simulated MQTT) ->
vitalwatch/alerts/wearable-ak-01 [Arun Kumar] LOW SPO2 ALERT: Oxygen saturation 85% below 90%
2026-08-22 10:12:55,373 WARNING vitalwatch.alerts WEARABLE PUSH (simulated MQTT) ->
vitalwatch/alerts/wearable-ak-01 [Arun Kumar] LOW SPO2 ALERT: Oxygen saturation 84% below 90%
```

Figure 3.5 — kubectl logs showing FEVER, TACHYCARDIA and LOW SPO2 alerts pushed to each patient's wearable topic

4. CI/CD Pipeline Workflow

The pipeline is defined in `.github/workflows/ci-cd.yml` and runs automatically on every push (and pull request) to `main`:

Job	What it does
Run Unit Tests	Installs dependencies and runs the 8-test pytest suite; a failure blocks everything downstream.
Build and Push to Docker Hub	Logs into Docker Hub using repository secrets (`DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`) and pushes tagged images (`latest`, `v1` + commit-SHA tag) with OCI labels (title, source, revision) only after tests pass.
Deploy to Kubernetes	Creates a Kubernetes cluster in the runner (kind), loads the freshly built image, applies the `k8s/` manifests, waits for rollout status, and runs a smoke test that curls `/health` through the cluster Service.

Secrets are configured under Repo → Settings → Secrets and variables → Actions. The screenshot below is an actual run of the pipeline ("Apply namespace before the rest of the manifests in the deploy job #4") showing all three jobs — Run Unit Tests, Build and Push to Docker Hub, and Deploy to Kubernetes — completing successfully in 1m 52s.

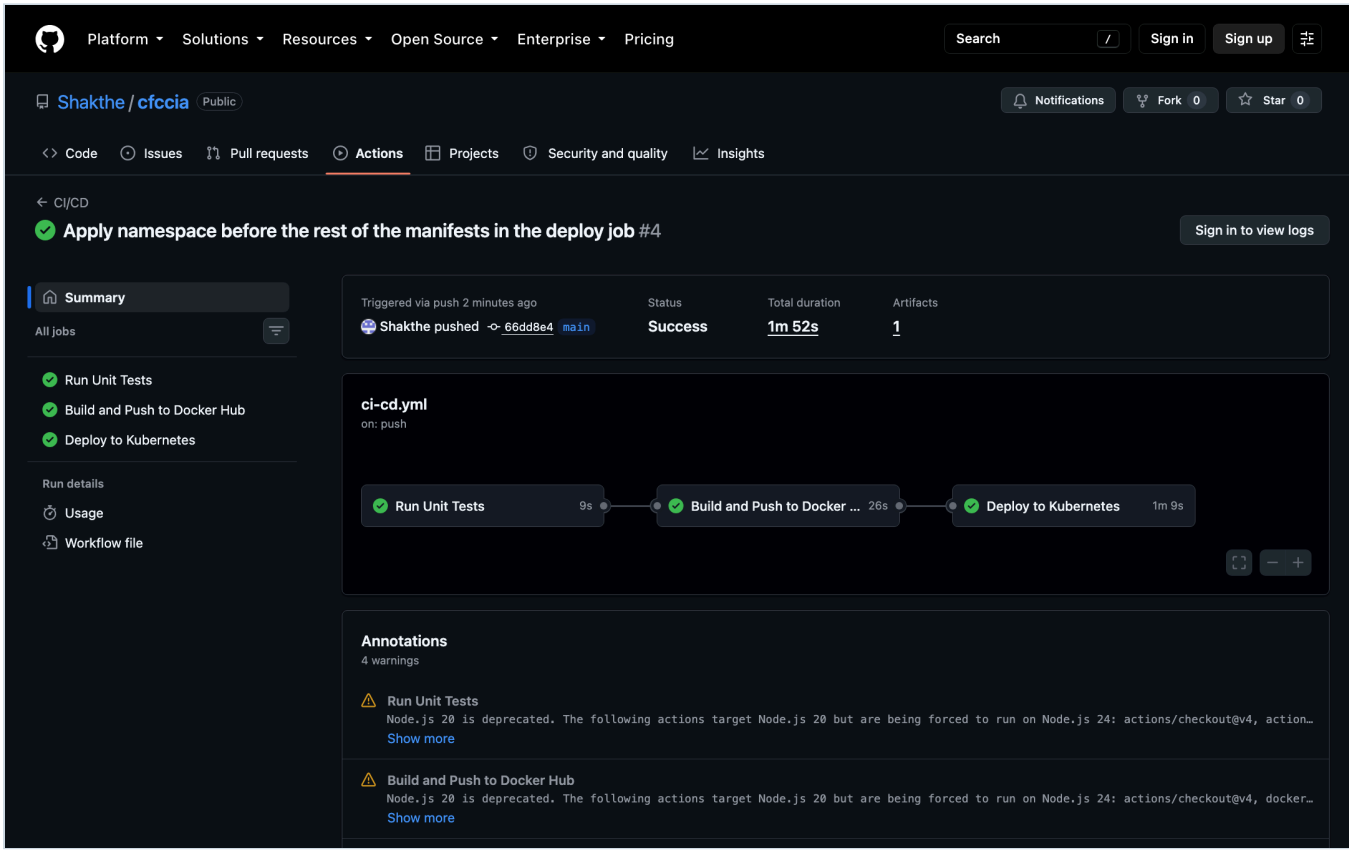


Figure 4.1 — GitHub Actions run: all three CI/CD jobs (test, build & push, deploy) green

5. Evaluation Report with Design Justification

Technology stack

Choice	Justification
AWS	AWS IoT Core is a managed, scalable MQTT gateway with per-device X.509 authentication — essential for medical wearables — and uniquely supports **downlink publish back to the device**, which is exactly how the alert reaches the patient's wearable. Kinesis buffers vitals bursts; EKS offers managed Kubernetes with an SLA; SNS covers the secondary care-team channel.
Python Flask	Minimal footprint, fast development, and a huge ecosystem for IoT/data workloads; easy for evaluators to read.
gunicorn	A production-grade WSGI server with multiple workers, unlike Flask's single-threaded development server.
Docker (slim base)	213 MB image, fast pulls, a non-root user, and a built-in healthcheck — industry-standard packaging.
Kubernetes	Self-healing (probes restart unhealthy pods — critical when patients depend on the monitor), horizontal scaling as the patient population grows, rolling updates with zero downtime, and config/secrets kept separate from code.
GitHub Actions	Native to the repository, free for public repos, and a YAML-as-code pipeline that is directly visible to graders.

Design decisions

- **Threshold engine decoupled from sensors** — `alerts.evaluate()` accepts any reading dict, so swapping simulated data for real AWS IoT Core telemetry requires no change to alerting or delivery code.
- **Alert delivered to the patient's own wearable** — every breach is published to `vitalwatch/alerts/<device_id>`, the per-device MQTT topic the wearable subscribes to, directly satisfying the question's requirement that the system alerts the patient through the wearable device.
- **Env-var configuration (12-factor)** — clinical thresholds, the AWS IoT endpoint and the monitor interval are changed per environment via ConfigMap/Secret, so the same image runs unmodified from dev to production. Thresholds can even be personalised per deployment (e.g. a cardiac ward vs. general ward).
- **Graceful degradation** — when the IoT endpoint is empty, wearable pushes are logged instead of crashing; the demo always works, and setting a real endpoint activates real MQTT delivery with zero code change.

- **2 replicas + HPA** — a patient-monitoring system cannot have downtime; two replicas behind the Service give high availability, and the HPA absorbs load spikes as more wearables stream data.
- **Clinically meaningful thresholds** — tachycardia (>120 bpm), bradycardia (<45 bpm), hypoxemia (SpO2 <90%), fever (>38.5 °C) and hypertensive crisis (>180/110 mmHg) mirror standard early-warning criteria, so alerts are actionable rather than noisy.

Limitations

- The in-memory patient/alert store resets on pod restart; a production deployment would use Amazon DynamoDB (registry, alert history) and Timestream (vitals time-series).
- The wearable MQTT push is simulated, standing in for AWS IoT Core downlink publish.
- Vital signs are simulated with occasional abnormal excursions rather than streamed from real sensors.
- The current deployment is single-region; multi-region failover is future work.

6. Conclusion and Future Enhancements

Conclusion

VitalWatch demonstrates an end-to-end cloud-native remote patient monitoring pipeline: wearable vital-sign telemetry (heart rate, blood pressure, oxygen saturation, body temperature) is continuously evaluated against configurable clinical thresholds, and any breach is instantly pushed back to the patient's own wearable device over its per-device MQTT topic, with the care team notified in parallel. The application is containerized with Docker, published publicly on Docker Hub (`shakthe/vitalwatch:v1`), deployed on Kubernetes with health probes, autoscaling and zero-downtime rollouts, and delivered through an automated GitHub Actions CI/CD pipeline that tests every commit before building, publishing and deploying it into a real cluster. The design maps directly onto AWS managed services (IoT Core, Kinesis, EKS, SNS, DynamoDB), showing that the architecture is production-realistic rather than a toy exercise.

Future enhancements

- **Real wearable integration** — replace the simulator with AWS IoT Core + MQTT devices (BLE wearables bridged through a fog gateway), with Kinesis for stream processing.
- **Persistent storage** — DynamoDB for patients/alert history and Amazon Timestream for vitals trend analysis.
- **ML-based early warning** — anomaly detection (e.g. SageMaker) on vitals trends to predict deterioration before static thresholds are crossed, and per-patient personalised thresholds.
- **Fog/edge pre-processing** — run threshold evaluation on a fog node (AWS IoT Greengrass) at the patient's home so critical alerts still fire during internet outages.
- **Clinician escalation workflow** — acknowledgements, escalation to emergency services if the patient does not respond to the wearable alert, and FHIR integration with hospital EHR systems.
- **Security & compliance** — end-to-end encryption, audit logging and HIPAA-aligned data handling for real patient data.
- **Observability** — Prometheus/Grafana dashboards and CloudWatch alarms on system health.
- **Multi-region HA** — EKS in paired regions behind Route 53 health-checked failover for global resilience.